

Part A – Simple Linear Regression (Manual Method)

```
exercises > simp_linReg_manual.ipynb > import numpy as np
Generate + Code + Markdown | Run All Restart Clear All Outputs View data Jupyter Variables Outline ...

import numpy as np
X=np.array([1,2,3,4,5,6])
Y=np.array([3,5,7,9,11,13])
mean_x = np.mean(X)
mean_y = np.mean(Y)
print("Mean of X: " + str(mean_x))
print("Mean of Y: " + str(mean_y))
a1 = np.sum((X-mean_x)*(Y-mean_y))/np.sum((X-mean_x)**2)
a0 = mean_y - a1*mean_x
print("A0: " + str(a0))
print("A1: " + str(a1))
print("Predicted value of Y for X=7: " + str(a0 + a1*7))

[2]

... Mean of X: 3.5
Mean of Y: 8.0
A0: 1.0
A1: 2.0
Predicted value of Y for X=7: 15.0
```

Part B – Simple Linear Regression using NumPy Matrix Method

```
exercises > simp_linReg_matrix.ipynb > import numpy as np
Generate + Code + Markdown | Run All Restart Clear All Outputs View data Jupyter Variables Outline ...

import numpy as np
X=np.array([[1,1],[1,2],[1,3],[1,4],[1,5],[1,6]])
Y=np.array([[3],[5],[7],[9],[11],[13]])
beta=np.linalg.inv(X.T.dot(X)).dot(X.T).dot(Y)
print("Beta values: " + str(beta))
print("A0: " + str(beta[0][0]))
print("A1: " + str(beta[1][0]))
print("Predicted value for x=7: " + str(beta[0][0] + beta[1][0]*7))

[3]

... Beta values: [[1.]
 [2.]]
A0: 0.9999999999999999
A1: 1.9999999999999999
Predicted value for x=7: 14.999999999999986
```

Part C – Multiple Linear Regression using Matrix Method

```
exercises > mul_linReg_matrix.ipynb > import numpy as np
Generate + Code + Markdown Run All Restart Clear All Outputs View data Jupyter Variables Outline ...

import numpy as np
X=np.array([[1,2,60],[1,3,65],[1,4,70],[1,5,75],[1,6,80],[1,7,85]])
Y=np.array([[40],[48],[55],[63],[70],[78]])
beta=np.linalg.pinv(X.T@X)@X.T@Y
print("Beta values: " + str(beta))
print("A0: " + str(beta[0][0]))
print("A1: " + str(beta[1][0]))
print("A2: " + str(beta[2][0]))
print("Predicted value for x1=8 and x2=90: " + str(beta[0][0] + beta[1][0]*8 + beta[2][0]*90))

[1]

... Beta values: [[-0.48860989]
 [ 4.98828187]
 [ 0.51091505]]
A0: -0.4886098857595276
A1: 4.988281868566833
A2: 0.5109150548580528
Predicted value for x1=8 and x2=90: 85.39999999999998
```

Part D – Simple Linear Regression using Scikit-learn

```
exercises > simp_linReg_sci-kit.ipynb > import numpy as np
Generate + Code + Markdown Run All Restart Clear All Outputs View data Jupyter Variables Outline ...

import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[1],[2],[3],[4],[5],[6]])
Y = np.array([3, 5, 7, 9, 11, 13])

model = LinearRegression()
model.fit(X, Y)

a0 = model.intercept_
a1 = model.coef_[0]

print("Intercept (a0):", a0)
print("Coefficient (a1):", a1)

x1 = 7
y1 = a0 + a1 * x1

print("Predicted value for X =", x1, ":", y1)

x2 = 10
y2 = a0 + a1 * x2

print("Predicted value for X =", x2, ":", y2)

[2]

... Intercept (a0): 1.0000000000000018
Coefficient (a1): 1.9999999999999996
Predicted value for X = 7 : 14.999999999999998
Predicted value for X = 10 : 21.0
```

Part E – Multiple Regression using Scikit-learn

```
exercises > mul_linReg_scikitpynb > import numpy as np
Generate + Code + Markdown | Run All Restart Clear All Outputs View data Jupyter Variables Outline ...

import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([[2, 60],[3, 65],[4, 70],[5, 75],[6, 80],[7, 85]
])
Y = np.array([40, 48, 55, 63, 70, 78])

model = LinearRegression()

model.fit(X, Y)

a0 = model.intercept_
a1 = model.coef_[0]
a2 = model.coef_[1]

print("Intercept (A0):", a0)
print("Coefficient for Study Hours (A1):", a1)
print("Coefficient for Attendance (A2):", a2)

study_hours = 8
attendance = 90

predicted_marks = a0 + (a1 * study_hours) + (a2 * attendance)

print("\nPrediction")
print("Study Hours =", study_hours)
print("Attendance =", attendance)
print("Predicted Marks =", predicted_marks)
```

[1]

```
... Intercept (A0): -47.47032967032969
Coefficient for Study Hours (A1): 0.29010989010989013
Coefficient for Attendance (A2): 1.4505494505494507

Prediction
Study Hours = 8
Attendance = 90
Predicted Marks = 85.4
```